

KINNAIRD COLLEGE FOR WOMEN
UNIVERSITY, LAHORE



COAL FINAL TERM PROJECT

Section B

Group Members

- Shafaq Rehman F24BSCS017
- Hajra Daniya Khan F24BSCS067
- IshmalKhalid F24BSCS037

Project Description – NotPlayable Game Menu (COAL Assembly Project)

This project is a **Game Menu System** created in **8086 Assembly Language**.

The goal of the project is to design a fully interactive **text-based menu interface** using BIOS interrupts, keyboard input, cursor control, and basic graphics characters.

The program displays a **main menu** with options:

1. **Play Game**
2. **Instructions**
3. **Levels**
4. **Settings**
5. **Exit**

The user can move between the options using the **UP** and **DOWN** arrow keys.

The currently selected option is highlighted with a special colored marker.

When the user presses **Enter**, the program opens a new screen (page) showing a title of the selected option and a message telling the user to press **ESC** to go back.

The program uses separate logic for:

- Drawing borders
- Changing colors
- Printing text at positions
- Handling keypress events
- Navigating between pages
- Returning to the main menu
- Exiting the program safe

What My Program Does

1. Clears the Screen & Sets Background Color

The program first switches to text mode and applies a custom background color using **INT 10h**.

2. Draws a Colorful Border Around the Screen

A full rectangular border is drawn using ASCII line characters.

Different sides of the border use different colors to make the interface look more attractive.

3. Displays a Welcome Title and Game Title

Two title messages appear at the top:

- *"Welcome to NotPlayable Game Menu"*
- *"FUN KIDS GAME"*

A decorative bar is drawn above and below the game title.

4. Shows the Main Menu with 5 Options

Each menu item is printed at a fixed location.

The currently selected option shows a **colored indicator block**, so the player always knows where the cursor is.

5. Supports Full Keyboard Navigation

The program reads input using **INT 16h**:

- **Up Arrow (48h)** → Move menu selection up
- **Down Arrow (50h)** → Move menu selection down
- **Enter** → Open the selected page
- **ESC** → Go back / Exit

If the selection reaches the top and the user presses UP, it wraps to the bottom (and vice-versa).

6. Opens New Pages for Each Option

When the user presses ENTER, a new page loads with:

- A big title bar
- The page heading (Play, Instructions, Levels, Settings)
- A message: “Press ESC to go back”

Each page is drawn using the same color-box style as the main menu.

7. ESC Returns to the Main Menu

No matter which page the user is on, pressing **ESC** sends them back to:

- Main Menu
- First selected option reset to index 0 (Play Game)

8. Exit Option Ends the Program

When the user selects EXIT → the program:

- Clears the screen
- Safely terminates using **INT 21h / 4Ch**

List of Interrupts Used

INT 10h (Video Services)

- INT 10h / AH = 00h — Set video mode
- INT 10h / AH = 0Bh — Set background/color palette
- INT 10h / AH = 02h — Set cursor position
- INT 10h / AH = 09h — Write character with attribute

INT 21h (DOS Services)

- INT 21h / AH = 09h — Print string
- INT 21h / AH = 4Ch — Exit program

INT 16h (Keyboard Services)

- INT 16h / AH = 00h — Read key

Key Features of the Program

Fully working interactive menu
Multiple page navigation
Keyboard input handling
Custom cursor positioning
Colorful UI using BIOS interrupts
Smooth user navigation
Easy return to main menu
Clean exit routine

Assembly Code:

```
[org 0x0100]
```

```
jmp start
```

```
; =====
```

```
; DATA SECTION
```

```
; =====
```

```
selected_idx:  dw 0
```

```
menu_count:    dw 5
```

```
current_page:  dw 0
```

```
; Menu strings
```

```
menu_play:     db 'PLAY GAME$'
```

```
menu_inst:     db 'INSTRUCTIONS$'
```

```
menu_levels:   db 'LEVELS$'
```

```
menu_settings: db 'SETTINGS$'
```

```
menu_exit:    db 'EXIT$'
title_welcome: db 'Welcome to NotPlayable Game Menu$'
title_main:   db 'FUN KIDS GAMES$'
title_play:   db 'PLAY MODE - Get Ready!$'
title_inst:   db 'HOW TO PLAY$'
title_levels: db 'SELECT YOUR LEVEL$'
title_settings: db 'GAME SETTINGS$'
back_msg:     db 'Press ESC to go back$'
```

```
; =====
; CLEAR SCREEN WITH COLOR
; =====
```

```
clrscr:
```

```
    push ax
    push bx
    mov ah, 0x00
    mov al, 0x03
    int 0x10
```

```
    mov ah, 0x0B
    mov bh, 0x00
    mov bl, 0x0B
    int 0x10
    pop bx
    pop ax
    ret
```

```
; =====
; PRINT STRING
; =====
```

```
print:
```

```
    push ax
    mov ah, 0x09
    int 0x21
    pop ax
    ret
```

```
; =====
```

```
; SET CURSOR POSITION
```

```
; =====
```

```
setcursor:
```

```
    push ax
    push bx
    mov ah, 0x02
    mov bh, 0
    int 0x10
    pop bx
    pop ax
    ret
```

```
; =====
```

```
; PRINT COLORED CHAR AT POSITION
```

```
; =====
```

```
printchar:
```

```
    push ax
    push bx
    push cx
    call setcursor
    mov ah, 0x09
    mov bh, 0
    mov cx, 1
    int 0x10
    pop cx
    pop bx
    pop ax
    ret
```

```
; =====
```

```
; PRINT AT POSITION
```

```
; =====
```

```
printat:
```

```
    call setcursor
    mov dx, si
    call print
    ret
```

```

; =====
; DRAW BORDER
; =====
draw_border:
    push ax
    push bx
    push cx
    push dx

    ; Top border
    mov dh, 0
    mov dl, 0
    mov cx, 80
    mov bl, 0x0E
    mov al, 196
.top:
    call printchar
    inc dl
    loop .top

    ; Bottom border
    mov dh, 24
    mov dl, 0
    mov cx, 80
    mov bl, 0x0E
    mov al, 196
.bottom:
    call printchar
    inc dl
    loop .bottom

    ; Side borders
    mov dl, 0
    mov dh, 1
    mov cx, 23
    mov bl, 0x0C
    mov al, 179
.left:

```



```
call printchar
inc dh
loop .left
```

```
mov dl, 79
mov dh, 1
mov cx, 23
mov bl, 0x0D
mov al, 179
```

```
.right:
```

```
call printchar
inc dh
loop .right
```

```
pop dx
pop cx
pop bx
pop ax
ret
```

```
; =====
; DRAW MAIN MENU
; =====
```

```
draw_menu:
```

```
call clrscr
call draw_border
```

```
; Welcome message
mov dh, 1
mov dl, 23
mov si, title_welcome
call printat
```

```
; Title box
mov dh, 3
mov dl, 32
call setcursor
mov ah, 0x09
```

```
mov al, 254
mov bh, 0
mov bl, 0x0E
mov cx, 16
int 0x10
```

```
mov dh, 4
mov dl, 32
mov si, title_main
call printat
```

```
mov dh, 5
mov dl, 32
call setcursor
mov ah, 0x09
mov al, 254
mov bh, 0
mov bl, 0x0E
mov cx, 16
int 0x10
```

```
; Get selected index
mov bx, [selected_idx]
```

```
; PLAY GAME
mov dh, 9
mov dl, 33
call setcursor
cmp bx, 0
jne .play_normal
mov ah, 0x09
mov al, 16
mov bh, 0
mov bl, 0x0C
mov cx, 1
int 0x10
.play_normal:
mov dh, 9
```

```
mov dl, 35
mov si, menu_play
call printat
```

```
; INSTRUCTIONS
```

```
mov dh, 11
mov dl, 33
call setcursor
cmp bx, 1
jne .inst_normal
mov ah, 0x09
mov al, 16
mov bh, 0
mov bl, 0x0C
mov cx, 1
int 0x10
```

```
.inst_normal:
mov dh, 11
mov dl, 35
mov si, menu_inst
call printat
```

```
; LEVELS
```

```
mov dh, 13
mov dl, 33
call setcursor
cmp bx, 2
jne .levels_normal
mov ah, 0x09
mov al, 16
mov bh, 0
mov bl, 0x0C
mov cx, 1
int 0x10
```

```
.levels_normal:
mov dh, 13
mov dl, 35
mov si, menu_levels
```

```
call printat
```

```
; SETTINGS
```

```
mov dh, 15
```

```
mov dl, 33
```

```
call setcursor
```

```
cmp bx, 3
```

```
jne .settings_normal
```

```
mov ah, 0x09
```

```
mov al, 16
```

```
mov bh, 0
```

```
mov bl, 0x0C
```

```
mov cx, 1
```

```
int 0x10
```

```
.settings_normal:
```

```
mov dh, 15
```

```
mov dl, 35
```

```
mov si, menu_settings
```

```
call printat
```

```
; EXIT
```

```
mov dh, 17
```

```
mov dl, 33
```

```
call setcursor
```

```
cmp bx, 4
```

```
jne .exit_normal
```

```
mov ah, 0x09
```

```
mov al, 16
```

```
mov bh, 0
```

```
mov bl, 0x0C
```

```
mov cx, 1
```

```
int 0x10
```

```
.exit_normal:
```

```
mov dh, 17
```

```
mov dl, 35
```

```
mov si, menu_exit
```

```
call printat
```

ret

; =====

; DRAW PAGE

; =====

draw_page:

call clrscr

call draw_border

; Title box

mov dh, 3

mov dl, 20

call setcursor

mov ah, 0x09

mov al, 254

mov bh, 0

mov bl, 0x0E

mov cx, 40

int 0x10

; Title

mov dh, 5

mov dl, 25

call printat

mov dh, 7

mov dl, 20

call setcursor

mov ah, 0x09

mov al, 254

mov bh, 0

mov bl, 0x0E

mov cx, 40

int 0x10

; Back message

mov dh, 20

mov dl, 26

```
call setcursor
mov ah, 0x09
mov al, 17
mov bh, 0
mov bl, 0x0A
mov cx, 1
int 0x10
```

```
mov dh, 20
mov dl, 28
mov si, back_msg
call printat
```

```
ret
```

```
; =====
; MAIN PROGRAM
; =====
```

```
start:
```

```
    call draw_menu
```

```
main_loop:
```

```
    mov ah, 0x00
    int 0x16
```

```
    mov cx, [current_page]
    cmp cx, 0
    jne handle_page
```

```
    cmp ah, 0x48
    je move_up
    cmp ah, 0x50
    je move_down
    cmp al, 13
    je select_item
    cmp al, 27
    je exit_program
    jmp main_loop
```

```
move_up:
    mov ax, [selected_idx]
    cmp ax, 0
    je wrap_down
    dec ax
    mov [selected_idx], ax
    call draw_menu
    jmp main_loop
```

```
wrap_down:
    mov ax, [menu_count]
    dec ax
    mov [selected_idx], ax
    call draw_menu
    jmp main_loop
```

```
move_down:
    mov ax, [selected_idx]
    mov cx, [menu_count]
    dec cx
    cmp ax, cx
    je wrap_up
    inc ax
    mov [selected_idx], ax
    call draw_menu
    jmp main_loop
```

```
wrap_up:
    mov word [selected_idx], 0
    call draw_menu
    jmp main_loop
```

```
select_item:
    mov ax, [selected_idx]
    cmp ax, 0
    je goto_play
    cmp ax, 1
```

```
je goto_inst
cmp ax, 2
je goto_levels
cmp ax, 3
je goto_settings
cmp ax, 4
je exit_program
jmp main_loop
```

```
goto_play:
    mov word [current_page], 1
    mov si, title_play
    call draw_page
    jmp main_loop
```

```
goto_inst:
    mov word [current_page], 2
    mov si, title_inst
    call draw_page
    jmp main_loop
```

```
goto_levels:
    mov word [current_page], 3
    mov si, title_levels
    call draw_page
    jmp main_loop
```

```
goto_settings:
    mov word [current_page], 4
    mov si, title_settings
    call draw_page
    jmp main_loop
```

```
handle_page:
    cmp al, 27
    jne main_loop
    mov word [current_page], 0
    mov word [selected_idx], 0
```



```
call draw_menu  
jmp main_loop
```

```
exit_program:  
call clrscr  
mov ah, 0x4C  
int 0x21
```